Infosys Senior Flutter & OTT Specialist Interview Mastery Guide

Candidate: Akshay Chandran | Target Role: Senior Mobile Application Developer (Flutter / OTT Specialist)

Part 1: Official Infosys Interview Scoring & Evaluation Rubrics Analysis

Based on the official Infosys interview evaluation sheets (Images 6 & 7), candidates are evaluated using two distinct scorecards on a **1.0 to 5.0 scale**.

1.1 Technical Matrix Benchmark (Image 6)

Assessment Metric	Infosys Passing Benchmark	What Evaluators Look For to Award 4.0+	How You Demonstrate This with AetherStream
Jetpack Compose / Flutter UI	**3.8 / 5.0**	Deep knowledge of rendering pipelines, element lifecycle, and zero-allocation CustomPainters.	Explaining how the 60 Hz timeline scrubber was decoupled via `ValueNotifier` and CustomPainter to maintain 60 FPS without GPU stalls.
MVI / MVVM / BLoC Architecture	**3.5 / 5.0**	Clear separation of Presentation, Domain, and Data layers with unidirectional data flow.	Feature-First Clean Architecture across 6 independent BLoCs (`PlayerBloc` , `CatalogBloc` , `TelemetryBloc` , etc.) with immutable states.
Kotlin / Dart Coroutines & Streams	**3.8 / 5.0**	Microtask queue, Event loop, StreamControllers, backpressure, and async memory management.	Managing ExoPlayer streams, periodic QoE sampling timers, and TCP socket reachability probing.
RxJava / Reactive Streams	**3.5 / 5.0**	Reactive operators (`switchMap` , `debounce` , `distinctUntilChanged`).	Solving search race conditions using `stream_transform` 's `debounceRestartable()` transformer.
GitHub Actions / CI/CD	**4.0 / 5.0**	Automated multi-stage build, lint, test with coverage, and artifact distribution pipelines.	`.github/workflows/ci_cd.yml` with automated static analysis, 8 unit/widget test suites with coverage, and multi-ABI APK compilation.
REST APIs & Network Resiliency	**3.5 / 5.0**	Interceptors, token refresh mutex locks, exponential backoff with jitter, and offline sync.	Transactional **Offline Outbox Pattern** with HTTP 206 partial content resumable downloads.
Dagger / Dependency Injection	**3.5 / 5.0**	Inversion of Control, test doubles, and clean repository injection.	Repository interfaces in the domain layer injected into BLoCs via `RepositoryProvider` and test mocks using `mocktail` .
Espresso & JUnit / Widget Testing	**4.2 / 5.0**	Writing hermetic unit, state, and responsive widget tests covering cross-platform breakpoints.	`test/widget_test.dart` containing 8 comprehensive test suites verifying cold-boot recovery, BLoCs, and mobile/TV layouts.
Agile & Scrum	**4.0 / 5.0**	Sprint planning, Definition of Done, story estimations, and PR code reviews.	Demonstrating production Git workflows, semantic commits, and issue tracking.

1.2 Behavioral & Leadership Matrix Benchmark (Image 7)

Competency Dimension	Benchmark	Senior-Level Response Strategy
Communication (Target: 3.5+)	Structured, concise, avoiding rambling. Frame every answer using the **STAR method** (Situation, Task, Action, Result).	
Technical Expertise (Target: 3.0+)	Grounding every concept in real video engineering mechanics (GOP, CMAF, DRM, ExoPlayer, ABR ladders).	
Technical Leadership (Target: 3.5+)	Guiding architectural decisions, establishing linter rules, and documenting engineering best practices.	
Delivery & Quality Ownership (Target: 3.5+)	Taking end-to-end responsibility: not just writing code, but profiling memory, preventing frame drops, and managing CI/CD releases.	
Problem Solving & Decision Making (Target: 3.5+)	Systematically isolating root causes (e.g., how the 1-second vs 4-second video seek stall was diagnosed and solved).	
Stakeholder / Customer Management (Target: 4.0+)	Explaining engineering trade-offs clearly to product managers (e.g., explaining why SSAI is preferred over CSAI despite backend complexity).	

Part 2: Exhaustive Solutions to All 42 Copilot Interview Questions

---

Section 1: Flutter and Dart

Q1: What is the difference between `StatelessWidget` and `StatefulWidget`?

- **StatelessWidget**: Immutable. Its visual configuration depends entirely on constructor arguments. Once built, it cannot dynamically alter its appearance unless its parent rebuilds it with new arguments. The framework creates and destroys it with minimal CPU overhead.
- **StatefulWidget**: Consists of two separate classes:
- The immutable `StatefulWidget` subclass (the blueprint).
- The mutable `State<T>` object that persists across widget rebuilds in the Element tree.
- **Senior OTT Insight**: In `AetherStream`, components like `MediaPosterCard` are pure `StatelessWidget`s for high-performance 60 FPS scrolling in large grids. In contrast, `AetherVideoPlayerView` is a `StatefulWidget` because it manages hardware texture controllers, ticker streams, and full-screen state that must outlive widget rebuilds.

Q2: Explain the lifecycle of a `StatefulWidget`. When do you use `initState`, `didUpdateWidget`, and `dispose`?

1. `createState()`: Creates the mutable state object.
2. `initState()`: Called exactly once when the state object is inserted into the tree. Use for initializing controllers, subscribing to streams, and starting animation tickers.
3. `didChangeDependencies()`: Invoked immediately after `initState` and whenever an inherited widget (e.g., `Theme`, `MediaQuery`, `BlocProvider`) changes.
4. `build()`: Returns the widget hierarchy. Must be pure, fast, and free of side-effects.

5. `didUpdateWidget(covariant T oldWidget)`: Called when the parent rebuilds and provides new configuration properties to this widget.
6. *OTT Example*: When the user selects a new movie, the player widget is updated without destroying its underlying state. `didUpdateWidget` checks `if (oldWidget.media.id != widget.media.id)` and re-initializes playback without recreating the state object.
7. `deactivate()`: Called when the widget is temporarily removed from the tree.
8. `dispose()`: Permanent removal. Essential for cancelling timers, closing stream subscriptions, and disposing native platform player controllers to avoid memory leaks.

Q3: What are `Future`, `Stream`, and `async`/`await` in Dart?

- `Future<T>`: Represents a single asynchronous computation that completes with a value or throws an error at a future point (e.g., fetching a JSON manifest from CloudFront).
- `Stream<T>`: A sequence of asynchronous events emitted over time. Streams can be single-subscription (e.g., network download chunks) or broadcast (e.g., continuous playhead position ticks).
- `async` / `await`: Declarative syntax over the Dart Event Loop that allows writing non-blocking asynchronous code in a sequential, readable structure.

Q4: How does null safety work? Explain `?`, `!`, and `late`.

- **Sound Null Safety**: Guarantees that non-nullable types can never contain `null` at compile time or runtime.
- `?` (Nullable Operator): Tells Dart that a variable or return value can hold a null value (e.g., `MediaItem? activeMedia`).
- `!` (Null Assertion Operator): Forces Dart to treat a nullable variable as non-null. Throws a runtime crash if it evaluates to `null`. In production code, avoid `!` and use null-coalescing (`??`) or pattern matching instead.
- `late`: Defers variable initialization until it is first accessed, asserting to the compiler that it will be initialized before usage (e.g., `late final VideoPlayerController _controller`).

Q5: What are widget keys, and when would you use them?

- Keys preserve state across widget tree restructuring by uniquely identifying elements of the same runtime type.
- **`ValueKey<T>`**: Identifies a widget by its underlying business ID. In `AetherStream`, the player uses: `dart key: ValueKey('${media.id}_${activeEpisode?.id}_${media.streamUrl}')` This forces Flutter to create a clean player instance whenever the stream URL or episode changes.
- **`GlobalKey`**: Uniquely identifies an element across the entire app, allowing access to its State (e.g., `ScaffoldMessengerState`, `NavigatorState`).
- **`ObjectKey`** / **`UniqueKey`**: Used for dynamic list animations and reordering items.

Q6: How would you structure a Flutter project to keep UI, business logic, and API integration separate?

- By implementing **Feature-First Clean Architecture**:
- **Presentation Layer**: Widgets, Pages, and BLoC/Cubit. Has no direct knowledge of API libraries or database schemas.
- **Domain Layer**: Pure Dart entities, value objects, and abstract repository contracts. Contains zero Flutter framework dependencies.

- **Data Layer**: DTOs, API clients (Dio/Http), local caching (SQLite/SharedPreferences), and repository implementations mapping raw JSON to domain models.

Section 2: BLoC/Cubit State Management

Q7: What is the difference between BLoC and Cubit? When would you choose each?

- **Cubit**: State management using functions. You call methods directly (`cubit.selectNetworkProfile(id)`) and call `emit(newState)` . Simple, lightweight, less boilerplate.
- *When to choose*: UI toggles, filters, simple CRUD operations (e.g., `AwsPipelineCubit` , `WatchlistCubit`).
- **BLoC**: Event-driven state management. Components communicate strictly by adding events (`bloc.add(Event())`), processed through event transformers (`debounceTime` , `restartable`).
- *When to choose*: Complex flows requiring event buffering, audit trails, concurrency control, or reactive streaming (e.g., `PlayerBloc` , `TelemetryBloc`).

Q8: How would you represent loading, success, empty, and error states?

Using Dart sealed classes with typed payloads:

```
sealed class CatalogState extends Equatable {
  const CatalogState();
}

final class CatalogInitial extends CatalogState { const CatalogInitial(); @override List<Object?> get props =
final class CatalogLoading extends CatalogState { const CatalogLoading(); @override List<Object?> get props =
final class CatalogLoaded extends CatalogState {
  final List<MediaItem> allItems;
  final String activeCategory;
  bool get isEmpty => allItems.isEmpty;
  const CatalogLoaded({required this.allItems, this.activeCategory = 'All'});
  @override List<Object?> get props => [allItems, activeCategory];
}
final class CatalogError extends CatalogState {
  final String message;
  final bool isNetworkFailure;
  const CatalogError({required this.message, this.isNetworkFailure = false});
  @override List<Object?> get props => [message, isNetworkFailure];
}
```

Q9: What is the difference between `BlocBuilder` , `BlocListener` , and `BlocConsumer` ?

- `BlocBuilder` : Rebuilds the UI based on state. Must be a pure function with **no side effects** (no navigation, no SnackBars).
- `BlocListener` : Does not render UI. Executes one-time side effects (navigation, analytics dispatch, SnackBars) when state changes.
- `BlocConsumer` : Combines both `builder` and `listener` into a single widget to avoid nesting.

Q10: How would you prevent unnecessary widget rebuilds?

1. Use `buildWhen` in `BlocBuilder` to filter state changes: `dart buildWhen: (prev, curr) => prev.activeMedia?.id != curr.activeMedia?.id`
2. Split large monolithic widgets into small `const` sub-widgets.

3. Decouple high-frequency tickers (e.g., player timecode) from the main UI using isolated `ValueListenableBuilder` or custom `RepaintBoundary` painters.

Q11: How would you manage shared state across multiple screens?

1. Provide BLoCs high in the widget tree using `MultiBlocProvider` above the `MaterialApp`.
2. Keep state persistent in shared services injected via repository interfaces.
3. Listen to cross-bloc state using `BlocListener` in the shell scaffold (e.g., when `PlayerBloc` transitions to mini-player mode, the shell automatically navigates back to the discovery tab).

Q12: How would you test a Cubit or BLoC that retrieves data from an API?

Using the `bloc_test` package and `mocktail`:

```
blocTest<CatalogBloc, CatalogState>(
  'emits [CatalogLoading, CatalogLoaded] when LoadCatalogRequested is added',
  build: () {
    when(() => mockRepo.getCatalogItems()).thenAnswer((_) async => mockItems);
    return CatalogBloc(repository: mockRepo);
  },
  act: (bloc) => bloc.add(const LoadCatalogRequested()),
  expect: () => [
    const CatalogLoading(),
    CatalogLoaded(allItems: mockItems),
  ],
  verify: (_) => verify(() => mockRepo.getCatalogItems()).called(1),
);
```

Section 3: Responsive UI and Reusable Components

Q13: What is the difference between responsive and adaptive UI?

- **Responsive UI**: The layout adjusts fluidly to different screen dimensions and aspect ratios (e.g., resizing columns from 2 to 4 as screen width expands).
- **Adaptive UI**: The application changes its input modality, navigation paradigms, and behavior depending on the target platform (e.g., touch navigation with bottom navigation bars on mobile vs. mouse hover and DPAD keyboard focus with `NavigationRail` on Desktop and Android TV).

Q14: When would you use `MediaQuery` versus `LayoutBuilder`?

- `MediaQuery.of(context)`: Provides global window dimensions, device pixel ratio, safe area padding, and system text scale factor. Use for root-level routing decisions and responsive breakpoints.
- `LayoutBuilder`: Provides the localized `BoxConstraints` (min/max width and height) passed down by the immediate parent widget. Use for individual self-adapting components (e.g., a card that switches from vertical to horizontal when width < 300px).

Q15: How would you build a screen that works on mobile, tablet, and web?

Create a centralized breakpoint helper (`AetherBreakpoints`) and an adaptive shell:

```
enum DeviceFormFactor { mobile, tablet, desktopTv }

class AetherBreakpoints {
  static DeviceFormFactor formFactorOf(BuildContext context) {
    final width = MediaQuery.sizeOf(context).width;
    if (width < 640) return DeviceFormFactor.mobile;
    if (width < 1080) return DeviceFormFactor.tablet;
    return DeviceFormFactor.desktopTv;
  }
}
```

- **Mobile**: Bottom `NavigationBar`, 2-column poster grid.
- **Tablet**: Compact `NavigationRail`, 3-4 column grid.
- **Desktop/Web**: Extended `NavigationRail`, 5-6 column grid, split-pane player studio.

Q16: How would you create a reusable component with different styles and states?

Using a component-driven pattern with clear configuration enums:

```
class MediaPosterCard extends StatelessWidget {
  final MediaItem item;
  final PosterCardVariant variant; // compact, standard, wideFeatured
  final VoidCallback onTap;
  ...
}
```

Isolate states (loading placeholder, image cached, error fallback, badge overlays) within the component.

Q17: How would you handle text scaling, orientation changes, and layout overflow?

1. Wrap text in `Text(..., maxLines: 2, overflow: TextOverflow.ellipsis)`.
2. Clamp system font scaling to prevent UI breakage: `dart MediaQuery.withClampedTextScaling(maxScaleFactor: 1.25, child: ...)`
3. Use `SingleChildScrollView`, `Wrap`, or `FittedBox` inside flexible columns.

Section 4: REST API Integration

Q18: How would you organize the API client, repository, and data models?

- **API Client**: Handles base URL, timeout configurations, auth interceptors, and raw HTTP calls.
- **Data Model (DTO)**: Contains `fromJson` and `toJson` serialization logic.
- **Repository**: Interfaces with the API client, caches responses locally, handles error mapping, and transforms DTOs into pure Domain entities.

Q19: How would you handle timeouts, failed requests, and retry behavior?

1. Configure explicit connect, send, and receive timeouts (e.g., 8–15 seconds).
2. Implement **Exponential Backoff with Jitter** for transient errors (HTTP 502/503/504, network drops): $$\text{Delay} = 2^{\text{attempt}} \times 500\text{ms} + \text{jitter}$$
3. Avoid retrying 4xx client errors (401, 403, 404).

Q20: How would you handle an expired access token and concurrent API requests?

Using an **Auth Interceptor with a Mutex / Queuing Lock**: 1. When a request returns HTTP 401 Unauthorized, pause outgoing requests. 2. If a refresh is already in flight, enqueue the request. 3. If no refresh is running, invoke the token refresh endpoint using the persisted refresh token. 4. On success, update the stored access token, update authorization headers, and replay all queued requests. 5. On failure, flush the auth store and emit a sign-out event.

Q21: How would you implement pagination without making duplicate requests?

Maintain an `isFetchingNextPage` guard and trigger requests via `ScrollController`:

```
void _onScroll() {
  if (_controller.position.pixels >= _controller.position.maxScrollExtent - 400) {
    if (!state.isFetchingNextPage && state.hasMorePages) {
      context.read<CatalogBloc>().add(const FetchNextCatalogPage());
    }
  }
}
```

Q22: How would you handle search when responses arrive in a different order from the requests?

By using BLoC's `restartable()` transformer (powered by Rx `switchMap`): * When the user types `A`, then `Ab`, then `Abc`, each new search event cancels the previous in-flight HTTP request. Only the response matching the latest query is emitted, eliminating race conditions.

Section 5: Cross-Platform and Smart TV

Q23: What platform-specific challenges have you encountered across Android, iOS, and web?

- **Web**: Autoplay policy restrictions (browsers block unmuted video playback without prior user interaction); CORS policies on HLS/DASH manifest fetching.
- **Android**: SurfaceProducer fence sync lockups on Android < 33 with Impeller; fragmented hardware codec profiles (HEVC Main10 vs baseline).
- **iOS**: AirPlay DRM handshake protocols; background audio session interruption policies.

Q24: How would you verify Flutter and plugin compatibility for a particular Smart TV platform?

1. Check Android TV compatibility: Ensure `leanback` launcher intent is declared and touch screen is marked non-required in `AndroidManifest.xml`: `xml <uses-feature android:name="android.hardware.touchscreen" android:required="false" /> <uses-feature android:name="android.software.leanback" android:required="true" />`
2. Verify native media decoding: Ensure hardware codecs support 60 FPS H.264/HEVC on low-powered TV SoCs.

Q25: How would you implement remote-control navigation and visible focus states?

Wrap interactive elements in `FocusableActionDetector` or `InkWell`: * Listen to `FocusNode.hasFocus`. * Render a distinctive glowing border, scaling effect ($1.08\times$), or accent highlight when focused.

Q26: How would you adapt a touch-based screen for keyboard, mouse, and TV remote input?

1. Implement `Shortcuts` and `Actions` mapping DPAD (Up, Down, Left, Right, Select/Enter, Back) to focus traversal.
2. Enable keyboard hover listeners (`InkWell(onHover: ...)`) for desktop and web cursors.

Q27: How would you test navigation, back-button behavior, and video playback across platforms?

1. Use `flutter test` with `tester.sendKeyEvent(LogicalKeyboardKey.select)` to simulate TV remotes.
2. Test Android hardware back button using `PopScope(canPop: false, onPopInvokedWithResult: ...)` to handle mini-player minimization.

Section 6: Performance, Testing, and CI/CD

Q28: How would you investigate scrolling lag on a screen containing many images?

1. Open Flutter DevTools Performance view and inspect the **Frame Rendering Chart** for jank bars (> 16.6ms).
2. Set explicit cache bounds on images: `dart ResizeImage(imageProvider, width: 300, height: 450)`
3. Wrap cards in `RepaintBoundary` to avoid repainting unchanged tiles when the list scrolls.

Q29: How would you identify memory leaks and clean up controllers or subscriptions?

1. Use the **DevTools Memory Profiler** to take heap snapshots before and after navigating back from the Player screen.
2. Look for retained instances of `VideoPlayerController`, `Timer`, or `StreamSubscription`.
3. Ensure every controller and timer is closed in the `dispose()` callback.

Q30: What is the difference between unit, widget, and integration tests?

- **Unit Test**: Tests a single class or function in isolation (e.g., Duration formatter, BLoC state transitions). Runs in seconds without UI.
- **Widget Test**: Tests a component in a headless Flutter environment. Verifies layout, user taps, and state binding.
- **Integration Test**: Runs the full application on a real device/emulator, verifying native platform channels, ExoPlayer decoding, and network communication.

Q31: What checks would you include in a Flutter CI/CD pipeline?

1. **Linter & Static Analysis**: `flutter analyze` with strict rules.
2. **Automated Test Execution**: `flutter test --coverage` ensuring > 80% coverage.
3. **Artifact Compilation**: `flutter build apk --release` and `flutter build web --release`.
4. **Artifact Archiving**: Uploading binaries securely to GitHub Artifacts / AWS S3 / Firebase App Distribution.

Q32: How would you manage dev, QA, and prod configurations without exposing secrets?

Using **Flutter Flavors / Compile-time environment variables**:

```
flutter build apk --dart-define=APP_ENV=prod --dart-define=API_KEY=$PROD_KEY
```

Store production credentials in GitHub Actions Secrets and inject them at build time.

Section 7: OTT, Streaming, Firebase, and AWS

Q33: Which video-player integrations have you worked with, and how did you select them?

- **Google ExoPlayer (Android) & AVPlayer (iOS)** via `video_player` / `media_kit`: Selected for production stability, hardware surface rendering, multi-audio/subtitle track management, and Widevine/FairPlay DRM support.

Q34: What are HLS, DASH, and adaptive bitrate streaming?

- **HLS (HTTP Live Streaming)**: Apple's protocol using `.m3u8` manifests and `.m4s` / `.ts` segments.
- **MPEG-DASH**: International standard protocol using `.mpd` XML manifests.
- **ABR**: Dynamically selects video chunk bitrates every 2 seconds based on measured network bandwidth, preventing buffering.

Q35: How would you handle buffering, playback errors, subtitles, and resume playback?

- **Buffering**: Display non-blocking shimmering indicator; pause telemetry timer.
- **Errors**: Retry fetching chunk manifests up to 3 times with exponential backoff before showing user-friendly recovery UI.
- **Subtitles**: Parse WebVTT cues synchronized with player timecode.
- **Resume Playback**: Hydrate persisted timestamp from disk; seek to offset if $\ge 15\text{s}$.

Q36: What video analytics events would you capture, and how would you avoid duplicate events?

- **Events**: `playback_start`, `first_frame_rendered` (TTFF), `bitrate_switched`, `rebuffer_started`, `rebuffer_ended`, `playback_complete`.
- **Deduplication**: Enforce state transition guards and assign unique `playback_session_id`s.

Q37: How have you used Firebase for auth, notifications, crash reporting, or analytics?

- **Firebase Auth**: User credentials and anonymous guest tokens.
- **Firebase Cloud Messaging (FCM)**: Push notifications for new releases and live match start alerts.
- **Crashlytics**: Real-time crash logging with custom keys (`user_tier`, `active_resolution`, `codec`).
- **Firebase Remote Config**: Dynamic feature flagging.

Q38: Explain the roles of MediaLive, MediaPackage, and CloudFront in live streaming.

- **MediaLive**: Ingests RTMP/SRT broadcast feeds and encodes them into multiple bitrate streams in real time.
- **MediaPackage**: Takes streams from MediaLive, packages them into HLS/DASH, and applies DRM encryption.
- **CloudFront**: Edge CDN caching video chunks worldwide to deliver them to millions of concurrent viewers with low latency.

Section 8: Practical Exercises

Exercise 39: API-Driven Screen with Cubit (Loading, Empty, Error, Retry, Pagination)

```
// 1. State Definition
sealed class MediaGridState extends Equatable {
  const MediaGridState();
}
class MediaGridLoading extends MediaGridState { const MediaGridLoading(); @override List<Object?> get props =
class MediaGridLoaded extends MediaGridState {
  final List<MediaItem> items;
  final bool isFetchingNextPage;
  final bool hasMore;
  const MediaGridLoaded({required this.items, this.isFetchingNextPage = false, this.hasMore = true});
  @override List<Object?> get props => [items, isFetchingNextPage, hasMore];
}
class MediaGridError extends MediaGridState {
  final String message;
  const MediaGridError(this.message);
  @override List<Object?> get props => [message];
}

// 2. Cubit Implementation
class MediaGridCubit extends Cubit<MediaGridState> {
  final CatalogRepository repository;
  int _page = 1;
  MediaGridCubit(this.repository) : super(const MediaGridLoading());

  Future<void> fetchItems() async {
    _page = 1;
    emit(const MediaGridLoading());
    try {
      final items = await repository.fetchPagedCatalog(page: _page);
      emit(MediaGridLoaded(items: items, hasMore: items.length >= 20));
    } catch (e) {
      emit(MediaGridError('Failed to load catalog: $e'));
    }
  }

  Future<void> fetchNextPage() async {
    final current = state;
    if (current is! MediaGridLoaded || current.isFetchingNextPage || !current.hasMore) return;
    emit(MediaGridLoaded(items: current.items, isFetchingNextPage: true, hasMore: current.hasMore));
    try {
      _page++;
      final nextItems = await repository.fetchPagedCatalog(page: _page);
      emit(MediaGridLoaded(items: [...current.items, ...nextItems], hasMore: nextItems.length >= 20));
    } catch (_) {
      emit(MediaGridLoaded(items: current.items, isFetchingNextPage: false, hasMore: false));
    }
  }
}
```

Exercise 40: Smart TV Grid Navigation with Focus and Selection

```
class TvMediaGridItem extends StatefulWidget {
  final MediaItem item;
  final VoidCallback onSelect;
  const TvMediaGridItem({super.key, required this.item, required this.onSelect});

  @override
  State<TvMediaGridItem> createState() => _TvMediaGridItemState();
}

class _TvMediaGridItemState extends State<TvMediaGridItem> {
  bool _isFocused = false;

  @override
  Widget build(BuildContext context) {
    return Focus(
      onFocusChange: (focused) => setState(() => _isFocused = focused),
      onKeyEvent: (node, event) {
        if (event.logicalKey == LogicalKeyboardKey.select ||
            event.logicalKey == LogicalKeyboardKey.enter) {
          widget.onSelect();
          return KeyEventResult.handled;
        }
        return KeyEventResult.ignored;
      },
      child: AnimatedScale(
        scale: _isFocused ? 1.08 : 1.0,
        duration: const Duration(milliseconds: 180),
        child: Container(
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(12),
            border: Border.all(
              color: _isFocused ? const Color(0xFF00F5D4) : Colors.transparent,
              width: 3.0,
            ),
            boxShadow: _isFocused
                ? [BoxShadow(color: const Color(0xFF00F5D4).withOpacity(0.4), blurRadius: 16)]
                : [],
          ),
          child: MediaPosterCard(item: widget.item, onTap: widget.onSelect),
        ),
      ),
    );
  }
}
```

Exercise 41: Debugging Race Conditions in Search Screen

```
EventTransformer<E> debounceRestartable<E>(Duration duration) {
  return (events, mapper) => events.debounceTime(duration).switchMap(mapper);
}

// In SearchBloc:
on<SearchQueryChanged>(
  _onSearchQueryChanged,
  transformer: debounceRestartable(const Duration(milliseconds: 300)),
);

Future<void> _onSearchQueryChanged(
  SearchQueryChanged event,
  Emitter<SearchState> emit,
) async {
  if (event.query.trim().isEmpty) {
    emit(const SearchEmpty());
    return;
  }
  emit(const SearchLoading());
  final results = await repository.search(event.query);
  emit(SearchLoaded(results));
}
```

Exercise 42: Universal Reusable Content Card (Mobile, Web, TV)

```
class UniversalContentCard extends StatelessWidget {
  final MediaItem? item;
  final bool isLoading;
  final bool isUnavailable;
  final VoidCallback? onTap;

  const UniversalContentCard({
    super.key,
    this.item,
    this.isLoading = false,
    this.isUnavailable = false,
    this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    if (isLoading) {
      return Container(
        decoration: BoxDecoration(
          color: Colors.white.withOpacity(0.08),
          borderRadius: BorderRadius.circular(10),
        ),
        child: const Center(child: CircularProgressIndicator(strokeWidth: 2)),
      );
    }

    if (isUnavailable || item == null) {
      return Container(
        decoration: BoxDecoration(
          color: Colors.black45,
          borderRadius: BorderRadius.circular(10),
          border: Border.all(color: Colors.white12),
        ),
        child: const Center(
          child: Text('Content Unavailable', style: TextStyle(color: Colors.white54, fontSize: 11)),
        ),
      );
    }

    return LayoutBuilder(
      builder: (context, constraints) {
        final isCompact = constraints.maxWidth < 140;
        return InkWell(
          onTap: onTap,
          borderRadius: BorderRadius.circular(10),
          child: ClipRRect(
            borderRadius: BorderRadius.circular(10),
            child: Stack(
              fit: StackFit.expand,
              children: [
                Image.network(
                  item!.posterUrl,
                  fit: BoxFit.cover,
                  errorBuilder: (_, __, ___) => const ColoredBox(color: Colors.grey),
                ),
                Positioned(
                  bottom: 0,
                  left: 0,
                  right: 0,
                  child: Container(
```

```
                    padding: EdgeInsets.all(isCompact ? 4 : 8),
                    decoration: const BoxDecoration(
                      gradient: LinearGradient(
                        begin: Alignment.bottomCenter,
                        end: Alignment.topCenter,
                        colors: [Colors.black87, Colors.transparent],
                      ),
                    ),
                    child: Text(
                      item!.title,
                      maxLines: 1,
                      overflow: TextOverflow.ellipsis,
                      style: TextStyle(
                        color: Colors.white,
                        fontSize: isCompact ? 10 : 13,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      );
    },
  );
 }
}
```